

Solving the yard crane scheduling problem with dynamic assignment of input/output points

Hongtao Wang^{a,*}, Fulgencia Villa^a, Eva Vallada^{a,c}, Rubén Ruiz^{a,b}

^a Grupo de Sistemas de Optimización Aplicada, Universitat Politècnica de València, Camino de Vera s/n, 46021, València, Spain

^b Amazon.com, Inc. P.O. Box 81226, Seattle WA, United States

^c ValgrAI, Spain

ARTICLE INFO

Dataset link: <https://github.com/HWangUPV/YCSP>

Keywords:

Yard crane scheduling problem
Heuristics
Input/output points
GRASP

ABSTRACT

This paper introduces an automatic yard crane scheduling problem with additional assignments of input/output (I/O) points at the block. I/O points are the buffer between the block and the rest of the terminal and containers are relocated from the block to the I/O points or vice-versa. The crane schedule therefore not only considers movements, storage and retrieval of containers, but must also be coordinated with the release and due times of containers at the I/O points, which are also limited and need to be assigned during the scheduling process. This results in a complex, but at the same time, more realistic problem. Two GRASP (Greedy Randomized Adaptive Search Procedure) heuristic approaches are proposed along with improvements to the solution construction and local search phases that are specifically tailored for this problem. The proposed algorithms are statistically calibrated and comprehensive experiments are designed to assess the performance of the method, including validation across small, medium, and large size instances. The experimental results show that the proposed methods are effective and competitive when compared to existing approaches. The GRASP manages to obtain an optimality average rate of 71.25% for small instances and finds 461 new best solutions in the 480 medium and large instances, with up to 200 containers, when compared to existing approaches for this problem.

1. Introduction

Containerized maritime transportation is a principal protagonist in and critical part of modern trade. Global containerized trade jumped from 45 million 20-foot equivalent unit (TEU) containers in 1996 to 165 million in 2021 (Sirimanne et al., 2022). Throughout port terminals all over the world, more than 857 million TEUs were handled in 2021, up from 542 million TEUs in 2010. This tremendous growth puts enormous pressure on both the sea as well as land sides of port terminals. To cope with this increased demand, capital investment in equipment (purchase and/or renovation) has proven to be effective (Gharehgozli et al., 2016). However, improvements in the operational efficiency of existing resources are often preferable to costly capital equipment investment.

Port terminals exist in two main layouts (Fig. 1): Asian and European terminal layouts. There are two salient differences between these: (1) The blocks of containers are parallel in the Asian and perpendicular in the European layouts with respect to the quay where ships are berthed; (2) Asian terminals allow trucks to pass the sideways lanes along the block so as to directly load containers whereas, input/output (I/O) points are often adopted in European layout to serve this purpose. Blocks are just clusters of containers that are organized in rows, bays

and tiers. Quay cranes (QCs) take containers in and out of the vessels and yard cranes (YCs) take containers in and out of the blocks, which serve as intermediate storage buffers between the vessels and the land transportation system (mainly trucks and trains). The I/O points, located on both sides of the block, allow for safer container loading by the YCs. They act as the transfer points between the block and the other parts of the system. Transportation systems within the port terminal, like Automated Guided Vehicles (AGVs), Straddle carriers (SCs) or trucks are used to move containers between the different areas. A large port terminal contains up to 100 YCs (in particular, according to Meade (2022), the largest port in the world, Shanghai, operated 108 rail-mounted gantry cranes in 2021). YCs, which have to carry out the majority of the TEU handling in terminals, are not only expensive to install and to operate, but they are also bulky and slow. As a result, they are often cited as a bottleneck for port performance (Ng and Mak, 2005). For these reasons, this paper studies efficient methods to improve YC efficiency in port terminals, and in particular, powerful algorithms to schedule YC movements under European terminal configuration.

* Corresponding author.

E-mail addresses: hwang8@doctor.upv.es (H. Wang), mfulvilju@eio.upv.es (F. Villa), evallada@eio.upv.es (E. Vallada), r Ruiz@eio.upv.es (R. Ruiz).

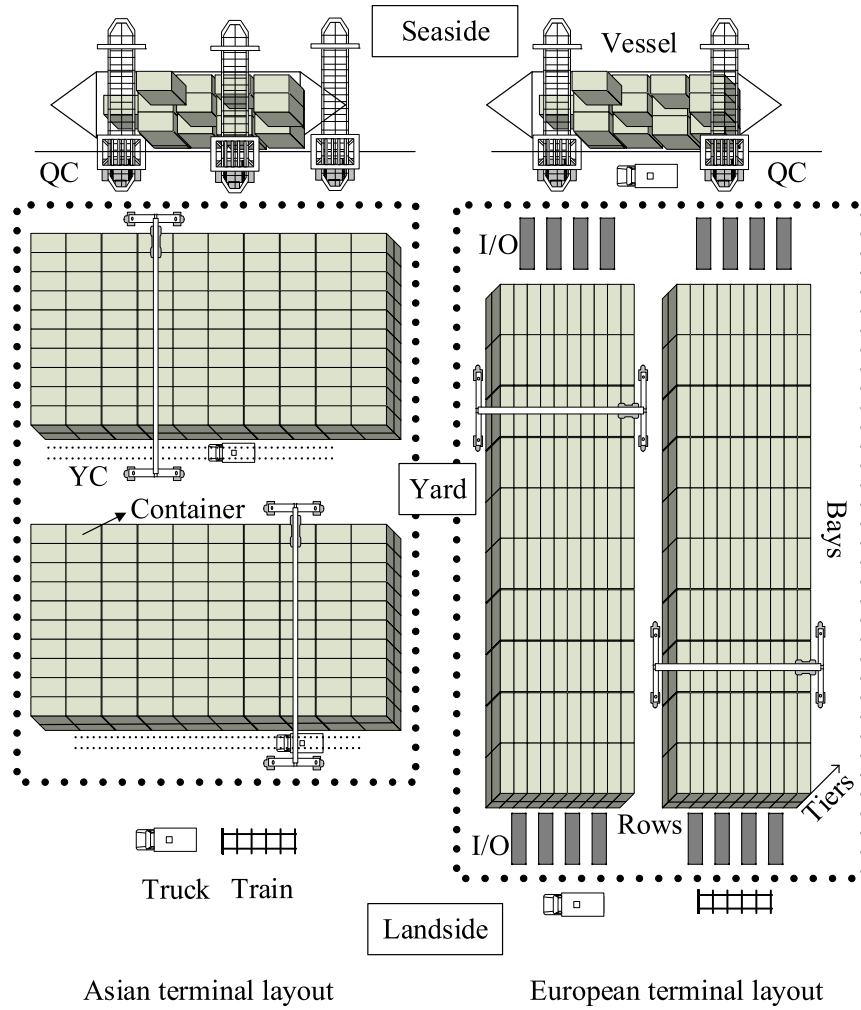


Fig. 1. Asian and European port terminal layouts.

YC operation optimization comprises multiple different problems. Among them, this paper studies the Single Yard Crane Scheduling Problem (YCSP). In the YCSP, a series of containers have to be handled in and out of the block with the aim of scheduling the container movements (referred to as requests) so that the total time to perform all requests is minimized. There are different types of operations. A retrieval operation entails first hoisting the spreader and then moving an unloaded YC from its current position to the position of the container we need to retrieve within the block. The spreader is lowered and the container hoisted. Then the YC moves to the destination position which could be within the block or to an I/O position and the container is unloaded. Storage operations are in some sense, reverse: The YC loads a container from one I/O point and unloads it in a given location within the block. Note that containers are always placed on the top of stacks and sometimes relocation (called reshuffles) of containers is needed. For more operational details, interested readers are referred to [Galle et al. \(2018\)](#), [Kizilay and Eliyi \(2021\)](#).

Let us focus on the consideration of the I/O points as a crucial component in YC operations. Each block has a limited number of I/O points where inbound/outbound containers are placed. However, the vast majority of the literature on the YCSP assumes infinite I/O points that are always available ([Ng and Mak, 2005](#); [Vis and Roodbergen, 2009](#); [Gharehgozli et al., 2014, 2019](#); [Hsu et al., 2021](#)). Of course, this is an unrealistic assumption, as only a limited number of I/O points are available. Storage and/or retrieval requests of containers cannot be realistically scheduled while ignoring the release times of storage

requests and due dates of retrieval requests. With I/O positions being a limited resource, an I/O-agnostic YC schedule might effectively run out of I/O positions for retrieval requests, for example, resulting in an unfeasible schedule in practice. Additionally, the same I/O points are used by all types of operations and congestion might appear where a given vehicle might not have empty I/O positions to place a container for a storage operation. Another situation might arise in which a YC could be deadlocked in a retrieval operation by not having a free I/O to place a container. In such a case, the YC would have to wait for a period of time until an I/O position is freed, resulting in unproductive waiting times.

This paper studies the YCSP in which I/O point assignment and scheduling is also considered. All YC operations are scheduled, together with the I/O assignments, so that the efficiency of the crane is maximized. To solve the problem efficiently, we propose a two-stage heuristic rule together with the Greedy Randomized Adaptive Search Procedure (GRASP) approach. In the first stage, containers are sequenced using constructive heuristic methods and rule-based heuristics to obtain a good initial solution. In the second stage, an adaptive I/O point assignment procedure is proposed to allocate the I/O point for each container and balance the occupancy in a given container sequence. Further improvements are proposed by a local search with a proxy objective to speed up calculations and a constructive method with I/O points reassignment. Two GRASP methods are formulated and statistically calibrated. An extensive computational study shows that the proposed GRASP produces optimal or near-optimal solutions for

small and medium instances. It also outperforms state-of-the-art Mixed Integer Programming (MIP) models and solvers as well as rule-based heuristics for large instances.

The rest of the paper is organized as follows. The related literature is reviewed in Section 2. The problem studied in this paper is formally described in Section 3. Section 4 details the proposed constructive heuristic and the local search while Section 5 details the GRASP metaheuristics, comprised of the initial constructive heuristic methods for container sequencing, I/O point assignment, local search and constructive phases. Comprehensive experiments are conducted in Section 6, and conclusions, discussion and future research are outlined in Section 7.

2. Literature review

Blocks served by a single crane are a common occurrence in port terminals and as a result have been investigated thoroughly. Ng and Mak (2005) studied the single YCSP in terminal ports with ready times as an extension of the MIP of Kim and Kim (1999) with proof of $\mathcal{NP}$ -completeness given by Narasimhan and Palekar (2002). As is common in the literature, Ng and Mak (2005) assume that each container request has fixed initial and final positions and there is no differentiation between storage and retrieval. Crane travel costs and request ready times are input data and the objective is to minimize the total waiting time. A Branch-and-bound (B&B) approach is proposed. The procedure takes advantage of the specific properties of the problem and is shown to be very effective.

Vis and Roodbergen (2009) extended the problem by considering storage and retrieval requests, where the destination and origin of each container are given. A Steiner Traveling Salesman Problem (TSP) based model is proposed and Dynamic Programming (DP) is used for its solution. However, the method must follow the one-lane crane assumption, which limits the application to modern ports where cross-line cranes have been adopted by most main ports (Gharehgozli et al., 2016).

More realistic single YCSPs have been studied. For example, Gharehgozli et al. (2014) consider I/O assignment to storage and retrieval requests. The authors use a two phase approach decomposition combining the solution from a relaxed assignment problem and branch and bound approaches. The objective is to minimize the total crane travel time. Gharehgozli et al. (2019) generalized the problem by integrating the stacking position allocation for storage movements from a set of open locations. Exact and heuristic algorithms are proposed for solving the problem.

An interesting YCSP extension has been studied by Galle et al. (2018) that also considers container relocations. Zheng et al. (2019) also tackle interesting generalizations. Starting from the problem studied in Ng and Mak (2005), they model the container retrieval release times stochastically. The scientific literature is littered with extensions of the YCSP with more than one crane: identical (twin) cranes or different (dual) cranes in which over-passing (stacking) can be allowed and therefore, different degrees of interference are required (Li et al., 2009; Vis and Carlo, 2010; Li et al., 2012; Zheng et al., 2018). From these extensions, twin cranes are the most similar to the single crane, as cranes cannot cross each other and operate on one side of the block separately. Gharehgozli et al. (2015) extend their previous study (Gharehgozli et al., 2014) to non over-passing twin cranes that must observe a safety separation distance. Other studies are Hu et al. (2016), Gharehgozli et al. (2017), Saini et al. (2017). Variants like triple stacking cranes (Dorndorf and Schneider, 2010), or cranes passing adjacent blocks are also presumed in Chen and Langevin (2011), Chu et al. (2019). Interested readers are referred to Speer and Fischer (2017) for a more in-depth discussion.

YCSPs are often integrated within broader operational port terminal problems. A few representative papers are Cao et al. (2010), Hsu et al. (2021) that consider yard trucks or Lee et al. (2011) that involves bay

allocation decisions. Kaveshgar and Huynh (2015) integrate quay and truck scheduling and Zhou et al. (2020) address vehicle positioning.

While exact approaches are commonly proposed to solve YCSPs, heuristics and metaheuristics are also very common in the literature and, in particular, for complex variants involving multiple cranes. Genetic Algorithms (GAs) are proposed in He et al. (2010), Chang et al. (2011), Chen and Langevin (2011), Liang et al. (2014), Hu et al. (2016), Zheng et al. (2018), Chu et al. (2019) and Yang and Jiang (2020), to name just a few. Simulated Annealing is used by Vis and Roodbergen (2009) while a Tabu Search is employed in Chen and Langevin (2011). Other techniques like Particle Swarm Optimization (PSO) are presented in Hsu et al. (2021). Local search is also often employed, as a stand-alone procedure or in combination with metaheuristics. An example is the Adaptive Large Neighborhood Search (ALNS) used by Gharehgozli et al. (2015).

The literature about YCSP involving I/O points is very scarce (Gharehgozli et al., 2014, 2015, 2019; Vallada et al., 2023). Gharehgozli et al. (2014) first introduce the I/O points in YCSPs. While Gharehgozli et al. assumed that the I/O points can be infinitely utilized with zero time cost, which is obviously too strong of an assumption as I/O points are limited in reality. In contrast, Vallada et al. (2023) have considered the availability of I/O points and the assignments coordinated with the release time or due time of containers. This results in more real-world I/O settings and thus is the one we adopt in this paper.

As can be seen, the YCSP is well researched, with several methodologies having been proposed for both single and multiple crane problems. However, it must be highlighted that exact approaches are only applicable to either less complex problem settings and/or small problem sizes. For example, the exact methods proposed in Gharehgozli et al. (2014) only consider fixed I/O points and ignore delays and congestion when assigning I/O points. Recently, Vallada et al. (2023) proposed a MIP model and simple heuristics for the problem we study in this paper. The results indicate that the MIP model, when paired with state-of-the-art solvers, is only capable of solving and proving the optimality of relatively small instances with up to 10 containers. As a result, in this paper we propose metaheuristic approaches based on GRASP for this problem.

3. Problem description

The YCSP consists of scheduling the movements of a yard crane in order to serve a set of container storage and retrieval requests with the objective of minimizing the total weighted costs involved in the process. We focus on a single block with R rows, B bays and T tiers. Moreover, the European layout is considered and there are two sets of input/output (I/O) points, one for seaside (I/O^S) and the other one for landside (I/O^L). During each movement, the crane is able to carry one container. The initial position of the crane is known (Ini) and the final position will be the destination of the last container in the sequence. Therefore, two decisions must be made to solve the problem: First, the order in which containers have to be processed by the crane and second, the assignment of an I/O point to each container. Each request is related to an origin location (O_c) and destination location (D_c). The set of container requests can be classified into four groups:

- C_1 (Sea to Yard): Containers arriving at the port on vessels to be stored in the block at a known position. Each container c has a known arrival time denoted as release time (r_c). Additionally, an I/O^S point on the seaside has to be assigned to c . The crane will pick up the container at an I/O^S (O_c) point, and place it in the destination (D_c) inside the block.
- C_2 (Land to Yard): Containers arriving at the port from the landside by train or truck to be stored in the block at a known position (D_c). There is also a release time for each container c denoted as r_c . In this case, an I/O^L point on the landside (O_c) has to be assigned to c .

- C_3 (Yard to Sea): Containers located in a known position (O_c) in the block to be retrieved and loaded onto a vessel. It is necessary to assign an I/O^S point on the seaside (D_c). Each container c has a due date at which it has to be delivered, denoted as d_c . Then, earliness and tardiness will be computed with respect to this date.
- C_4 (Yard to Land): Containers located in a known position (O_c) of the block to be retrieved and loaded onto a train or truck. In this case, an I/O^L point on the landside (D_c) is to be assigned. Each container c has a date on which the train or truck arrives, denoted as e_c . The container cannot be delivered before e_c as we are considering in this case, the I/O^L is the actual position in the train or truck where the container is loaded.

The container set is $C = \bigcup_{i=1}^4 C_i$ consisting of n containers. Fig. 2 shows the handling procedure for two container requests (c and c'). More specifically, the crane starts at position Ini with an empty movement from its current position to the initial position of c (O_c), then it loads and moves c to its destination (D_c). As is common in the literature (Galle et al., 2018; Speer and Fischer, 2017), the speeds of the gantry, trolley and hoisting crane are often considered as 1, so distance is equal to time. Positions of containers and I/O points are fixed and given with coordinates (x, y, z) before delivery, located at row $x \geq 0$, bay $y \geq 0$, and tier $z \geq 0$ in the yard block. Since the crane can always move vertically in rows with its trolley simultaneously moving, a pairwise Chebyshev distance is commonly used in the literature (Gharehgozli et al., 2014, 2019). The pairwise cost $t_{i,j}$ between locations i and j is defined as,

$$t_{i,j} = |z_i - (T + 1)| + \max\{|x_i - x_j|, |y_i - y_j|\} + |(T + 1) - z_j| + 2Res_{time} \quad (1)$$

where Res_{time} is the constant cost of relocation needed when a container is stacked in a middle tier and requires a reshuffle before the loading move. Note that the first and third terms of expression (1) are related to the crane going up and going down respectively. The crane starts the first movement at the top, so for the initial crane position, the z coordinate is $T + 1$ and then the first term of (1) will be zero. The example in Table 1 of Section 4.1 illustrates the use of expression (1) to obtain the final crane times for the objective function. According to Fig. 2, the crane starts an empty move at position Ini with a coordinate $(x_{Ini}, y_{Ini}, T + 1)$. Then, the crane moves to the origin (O_c) with coordinates (x_j, y_j, z_j) and loads the container c . The time needed for this loaded movement ($t_{Ini,j}$) is computed according to expression (1). After this, the crane moves to destination (D_c) with coordinates (x_k, y_k, z_k) for container c , using expression (1) to compute the time ($t_{j,k}$). Note that the flow of movements of each request (c and c') can be followed through the arrows, including the hoisting and lowering of the crane.

The objective of the problem is to find a sequence of container requests that minimizes the total weighted cost according to expression (2). There is a total of two costs: delays and congestion, which are penalized with weights w_c^R and w_c^C , respectively. Delay of requests can be caused by containers not arriving on time and congestion in I/O points which results in containers having to wait for I/O points to be free. Both delays and congestion are related to the r_c or e_c of the corresponding container c . Especially for $c \in C_3$, the container is possible to be loaded before the due date d_c resulting in earliness which can be measured as congestion since an I/O point is blocked until d_c . The objective function is detailed in expression (2). More details can be found in Vallada et al. (2023), where two mathematical models for this very same problem are also available.

$$\begin{aligned} \text{Min} \quad & \sum_{i=1}^2 \sum_{c \in C_i} (w_i^R (FC_c - r_c) + w_i^C (SO_c - r_c)) \\ & + \sum_{c \in C_3} (w_3^R (FO_c - d_c) + w_3^C E_c) \\ & + \sum_{c \in C_4} (w_4^R (FO_c - e_c) + w_4^C (SO_c - e_c)) \end{aligned} \quad (2)$$

FC_c is the final crane time for container c (the time at which the crane has finished placing container c in its final destination), SO_c and FO_c are the starting and final occupation times for container c at the I/O point, respectively, and E_c is the earliness for container $c \in C_3$. For storage/incoming containers, $c \in C_1 \cup C_2$ (containers entering the block), two terms are computed. The first one is related to the delay with respect to r_c ($FC_c - r_c$). The second is due to the I/O congestion, that is, there is no available I/O point when the container arrives at r_c and it has to wait until SO_c ($SO_c - r_c$). For containers $c \in C_3$, delays are computed as the difference between the time when the container leaves the I/O point (FO_c) and its due date (d_c). Moreover, congestion might result from assigning an I/O point before its due date d_c , which in turn generates earliness (E_c). Note that for containers $c \in C_3$, only one of the terms will be added, as it is not possible to be tardy and early at the same time. Finally, for containers $c \in C_4$, the delay is computed as the difference between the arrival time of the container at the I/O point (FO_c) and the arrival time of the vehicle (e_c). Note that in this case, $FO_c = FC_c$. The congestion is calculated in the same way as for containers $c \in C_1 \cup C_2$. Using the pairwise cost in Eq. (1) and the movements shown in Fig. 2, time variables of container c (the corresponding r_c , d_c and e_c values), FC_c , SO_c , FO_c and E_c , are calculated based on the state of the yard crane and I/O points. This denotes that the crane starts from the current position (initial position or last destination) ini at time T_{now} . The origin and the destination positions of container c are denoted as j and k respectively. Moreover, the chosen I/O point for c was only released by the previous request at time T_o . With all this clarified, the different terms of expression (2) are calculated as follows:

- Storage containers $c \in C_1 \cup C_2$. Given the state of the I/O and crane, we have that $SO_c = \max(r_c, T_o)$, and $FO_c = \max(T_{now} + t_{ini,j}, r_c)$. In this way, $FC_c = FO_c + t_{j,k}$.
- Retrieval containers $c \in C_3$. In this case, $FC_c = SO_c = \max(T_{now} + t_{ini,j} + t_{j,k}, T_o)$, and $FO_c = \max(d_c, FC_c)$. Finally, we define $E_c = \max(d_c - FC_c, 0)$.
- Retrieval containers $c \in C_4$. Since the loading at the I/O point starts after e_c , we have $SO_c = \max(e_c, T_o)$, and $FC_c = FO_c = \max(T_{now} + t_{ini,j} + t_{j,k}, e_c)$.

The example in Table 1 of Section 4.1 illustrates the use of expression (1) to obtain the final crane times for the objective function.

4. Constructive heuristic and local search procedure

We first present a two-stage heuristic, to be followed by a local search. The intended use of the heuristic is to provide an initial solution for the proposed GRASP metaheuristic as well as to serve the constructive phase at each iteration in the same GRASP. In the first step of the heuristic, we permute a sequence of containers. The second step carries out the I/O point assignment and calculates all times for each container. The following sections detail these two steps and the local search procedure.

4.1. First step: generation of a sequence of containers

A dispatching rule, referred to as the Simulated End-time Rule (*SETR*), is proposed to permute the sequence of containers. We start from an empty sequence and consider, at each iteration, all non-sequenced containers where a value according to expression (3) is computed. The container with the minimum η_c value is selected and included in the sequence. This means that

$$\eta_c = FC_c / w_i^R. \quad (3)$$

Recall that FC_c is an estimation of the final crane time for container c . It is just an estimation since an I/O point is required to compute it and these are not assigned until the second step. Therefore, an average

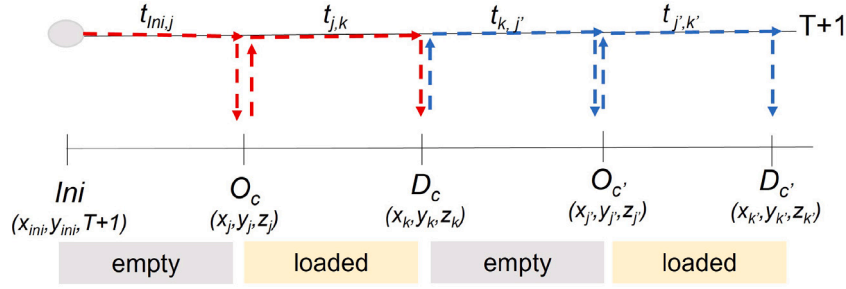


Fig. 2. Movement of the crane for two container requests.

Table 1

Input data for the SETR rule application example.

	c_1	c_2	c_3	c_4
Type	C_1	C_2	C_3	C_4
Time Parameter	4	3	1	2
w_i^R	1	3	2	4
Location (x, y, z)	(1, 7, 4)	(2, 3, 2)	(3, 5, 1)	(4, 3, 2)

Table 2

First container selection for the example in the SETR rule.

	c_1	c_2	c_3	c_4
FC_c	18	83	20	49
$\eta_c = FC_c / w_i^R$	18/1 = 18	83/3 = 27.7	20/2 = 10	49/4 = 12.25

(middle-placed) I/O point from the seaside/landside is assigned to all containers as an approximation.

Table 1 shows an example with 4 containers in a block with 4 tiers ($T = 4$). The time parameter corresponds to the containers' r_c , d_c or e_c , depending on their types. T_{now} is 0 in the beginning. For this example, we consider that the crane is initially located at (1, 2, 5) and the locations of the sea and land middle-placed I/O points are (1, 0, 1) and (1, 40, 2) respectively. Table 2 provides the calculated FC_c and η_c values for each container. We can detail the calculation for FC_{c_1} (a storage container), as follows: $FC_{c_1} = \max(T_{now} + t_{ini,j} + t_{j,k}; r_c + t_{j,k}) = \max(0 + 6 + 12; 4 + 12) = 18$, where $t_{ini,j} = |5 - 5| + \max\{|1 - 1|; |2 - 0|\} + |5 - 1| = 6$, and $t_{j,k} = |1 - 5| + \max\{|1 - 1|; |0 - 7|\} + |5 - 4| = 12$.

All calculated FC_c and η_c values are depicted in Table 2. It turns out that c_3 has the lowest η_c and is chosen as the first container in the sequence, resulting in $T_{now} = FC_{c_3} = 20$. The current position of the crane is the seaside I/O point at (1, 0, 1). From this position, we calculate the η_c again for the next container in the sequence. The final SETR sequence for the example is $\{c_3, c_4, c_2, c_1\}$. This first step of the heuristic does $\frac{n(n+1)}{2} - 1$ calculations and therefore has a computational complexity of $\mathcal{O}(n^2)$ for a total of n containers.

4.2. Second step: I/O assignment and timing

Once the sequence of containers is obtained, the next step is to assign I/O points for each container and compute container times for the objective function (expression (2)). The idea of the I/O point assignment is to allocate the best available I/O point for each container. All I/O points are examined with availability to determine if the I/O point is free at the time of the relevant time parameter, i.e., the release time ($r_c, c \in C_1 \cup C_2$), due date ($d_c, c \in C_3$) or vehicle arriving time ($e_c, c \in C_4$). In the extreme case where no I/O point is available, the one released at the earliest time is used. If multiple I/O points are available, the best one is chosen according to the time cost of subroutes for the different types of containers. The movements are divided into subroutes so that each one starts from the current location of the crane and ends at a container origin or destination in the yard. Subsequently, all I/O points needed during the subroute must be assigned. The

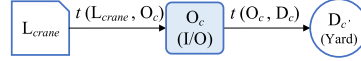
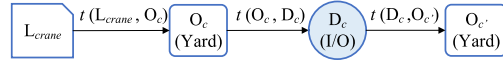
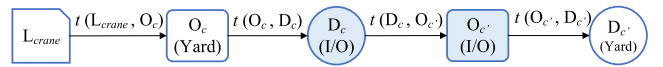
Case 1: $c \in \{C_1, C_2\}$ Case 2: $c \in \{C_3, C_4\}$ and $c' \in \{C_3, C_4\}$ Case 3: $c \in \{C_3, C_4\}$ and $c' \in \{C_1, C_2\}$ 

Fig. 3. Assigning I/O points in a sequence depending on the type of container.

following subroutes are calculated in the same way until completing the container sequence. Fig. 3 illustrates the different subroutes with the origin and destination locations, depending on the type of containers. We introduce the following notations: current location of the crane (L_{crane}), origin of container c (O_c) and destination (D_c). The arrows in Fig. 3 represent the time cost between two connected locations using Eq. (1). In Fig. 3, only one I/O (solid color square or circle) is needed for cases 1 and 2. Yet, the subroute of case 3 needs two I/O points for the assignment. The I/O assigned corresponds to the one (or two) available that minimize the total time of the subroute (the sum of all $t_{i,j}$ involved). In this way, all available I/O points (for cases 1 and 2) or I/O pairs (for case 3) should be considered. Once the I/O points are assigned, the starting and finish time of I/O occupation (SO_c and FO_c), as well as that of crane movement (SC_c and FC_c), can be calculated as well as the objective function (expression (2)) in Section 3.

The same instance in Table 1 is used to illustrate the I/O assignment with the SETR sequence, $\{c_3, c_4, c_2, c_1\}$. Let us suppose that there are an equal number of 4 I/O points $S1$ to $S4$ at $(s, 0, 1), s = \{1, 2, 3, 4\}$ in the seaside and $L1$ to $L4$ at $(l, 40, 2), l = \{1, 2, 3, 4\}$ in the landside. To load c_3 (type C_3), the crane starts from the initial position (L_{crane}) $Ini = (1, 2, 5)$ at time 0 to $O_{c_3}(3, 5, 1)$. At time $d_{c_3} = 1$, all $S1$ to $S4$ are available, and then c_3 is first delivered at O_{c_3} at $t_{Ini, O_{c_3}} = 7$. The best I/O point $S1$ is selected from $S1$ to $S4$ with subroute cost $t_{Ini, O_{c_3}} + t_{O_{c_3}, S1} + t_{S1, O_{c_4}} = 7 + 13 + 10 = 30$. In this case, $SC_{c_3} = 0 + t_{Ini, O_{c_3}} = 7$, $FC_{c_3} = SO_{c_3} = \max\{0 + 7 + 13; 1\} = 20$, $E_{c_3} = \max\{1 - 20; 0\} = 0$, and $FO_{c_3} = \max\{20; 1\} = 20$ with the calculations detailed in Section 3. Table 3 lists all the calculated terms of the objective function (2).

A comparison with a random or static I/O assignment was carried out in order to compare the effect of the dynamic assignment. Two kinds of pre-fixed I/O points assigned to each container were designed:

- RandIO: a random I/O point at the corresponding side;
- NearestIO: the nearest I/O point to the container's yard position at the corresponding side.

Table 3

Values for the objective function terms after the assignment of I/O points.

c	I/O	SC_c	FC_c	SO_c	FO_c
c_3	$S1$	7	20	20	20
c_4	$L1$	20	73	73	73
c_2	$L2$	79	122	3	79
c_1	$S2$	132	144	4	132

Additional experiments were carried out by only replacing the proposed dynamic assignment with the new two types of pre-fixed I/O points. The results proved that the proposed dynamic assignment is vastly superior to random or nearest I/O fixed assignments. The conclusion from the experiment was expected and follows intuition, since the proposed dynamic I/O assignment method is able to dynamically adjust the occupancy of I/O points, which is beneficial when decreasing the congestion at I/O points for the objective function.

4.3. Local search

Local search methods are widely used in optimization problems, including YCSP, in order to improve the quality of solutions (see Lim et al., 2004; He et al., 2015a,b; Galle et al., 2018). In essence, local search involves transitioning from one solution to another better one (a neighbor) in the solution space. This process is repeated following a certain order within a search loop. In algorithm 1, a local search method is proposed for the yard crane scheduling problem. We employ an insertion neighborhood for the sequence of containers with a best improvement strategy where each container is extracted and then reinserted into all positions in the sequence. However, reassigning I/O points proved computationally costly and a proxy function is employed. It is well known that local search should be as fast as possible so that large neighborhoods can be explored in little time. After all, the intention is to use the proposed local search within the proposed GRASP. Therefore, in order to speed up the evaluation of insertions, a proxy objective function is proposed (pf). The local search starts from an initial solution π , consisting of a sequence and an I/O assignment. The main loop consists of inserting all containers into all positions of the sequence and evaluating each solution using the proxy function. This proxy is an estimation of the real objective function f of expression (2). The only difference between pf and f is that pf does not require the I/O assignment. Instead, pf inherits the originally assigned I/O points for all containers in the original sequence. Note that this original assignment might no longer be feasible, but feasibility is not required to guide the local search. The final feasible solution is calculated when deciding if the new solution is to be accepted (line 11). The β best insertions are saved in a $TopBest$ set and only these β solutions are evaluated with the real objective function (f , which corresponds to expression (2), that is, including the I/O assignment and timing for the new sequence (Section 4.2) in order to obtain a feasible solution. The output of the local search method will be the best of these β solutions. Thanks to the use of the proxy function, for a given sequence with n containers and m I/O points, the complexity of one pass of the proposed local search decreases from $\mathcal{O}(n^2m^n)$ to $\mathcal{O}(n^2\beta m)$.

5. GRASP metaheuristic

GRASP (Feo and Resende, 1995) is a very well known multi-start metaheuristic with a vast amount of literature, applications and variants. GRASP performs especially well in cases where we can exploit a constructive heuristic that can be adapted to produce different solutions on each run. GRASP has been successfully applied to other crane scheduling problems (Kim and Park, 2004; Jung et al., 2006). For a more recent review of GRASP, the interested reader is referred to Resende and Ribeiro (2016). Algorithm 2 outlines the proposed GRASP to solve the YCSP. The algorithm starts by generating an initial solution

Algorithm 1 : Local search

```

1: Input:  $\pi$ ; Output:  $\pi_{best}$ 
2:  $\pi_{best} := \pi$ 
3:  $improvement := \text{true}$ 
4: while  $improvement == \text{true}$  do
5:    $TopBest := \emptyset$ 
6:    $improvement := \text{false}$ 
7:   for  $i := 1$  to  $n$  do
8:     Generate set  $TopBest$  with the best  $\beta$  insertions of container
        $i$  in all positions such that  $pf(TopBest)$  is minimized
9:     for  $b := 1$  to  $\beta$  do
10:      if  $f(TopBest(b)) < f(\pi_{best})$  then
11:         $improvement := \text{true}$ 
12:         $\pi_{best} := TopBest(b)$ 
13:       $\pi := \pi_{best}$ 

```

(π) with the deterministic constructive heuristic given in Section 4. The best initial solution (π_{best}) is obtained after applying the proposed local search algorithm to π . From this point, the main loop of GRASP starts, where during each iteration, a new solution is constructed in the constructive phase, detailed in the next section that is based on the heuristic of Section 4. This solution is later improved by the local search and finally, it is checked to see if it is a new best solution. Using different constructive methods, two GRASP methods are proposed in the following sections.

Algorithm 2 : GRASP metaheuristic

```

1: Output:  $\pi_{best}$ 
2:  $\pi := \text{Constructive heuristic}()$ 
3:  $\pi_{best} := \text{Local search}(\pi)$ 
4: while  $Time < MaximumTime$  do
5:    $\pi := \text{Constructive phase}()$ 
6:    $\pi' := \text{Local search}(\pi)$ 
7:   if  $f(\pi') < f(\pi_{best})$  then
8:      $\pi_{best} := \pi'$ 

```

5.1. GRASP constructive method 1

Algorithm 3 details the structure of the first constructive phase of the GRASP. It is iterative, greedy and randomized. Here, a first container is randomly selected. In the main loop, the remaining containers are ordered according to the *SETR* rule (Section 4.1) and the top/best α containers are selected to be part of the Restrictive Candidate List (RCL). One of the containers on the RCL is randomly selected and appended to the container sequence. The process is repeated until a complete sequence of containers is obtained. At this point, the I/O assignment and timing are computed according to Section 4.2. This GRASP method is referred to as *GR1*.

Algorithm 3 : GRASP constructive method 1

```

1: Output:  $\pi$ 
2:  $\pi = \emptyset$ 
3: Randomly append container  $c_0 \in C$  to  $\pi$ ,  $C \leftarrow C \setminus c_0$ 
4: while  $C \neq \emptyset$  do
5:   Sort all containers  $c \in C$  in non-decreasing order of  $\eta_c$ 
     (expression (3))
6:    $RCL :=$  Pick the first  $\alpha$  containers as the candidates of the  $RCL$ 
7:    $\pi :=$  Append one random container  $r \in RCL$  to the end of  $\pi$ 
8:    $C \leftarrow C \setminus r$ 
9: I/O assignment and Timing ( $\pi$ )

```

5.2. GRASP constructive method 2

One more GRASP algorithm, *GR2*, is introduced with a different constructive method described in Algorithm 4. Instead of assigning the I/O points for the final sequence, *GR2* integrates both container sequencing and I/O decisions. More precisely, we maintain the original I/O during container sequencing and then reassign the I/O point for each newly added container by exploring alternative I/O options on the same side. The I/O assignment and timing of Section 4.2 are performed to update I/O points if the solution can be improved. Note that the additional adjustment of I/O points results in additional complexity, while it significantly improves the quality of the solutions obtained, as demonstrated in later sections.

Algorithm 4 : GRASP constructive method 2

```

1: Input: the incumbent solution  $\pi_0$ ; Output:  $\pi$ 
2:  $\pi = \emptyset$ 
3:  $C \leftarrow$  Preserve the I/O point for each container from  $\pi_0$ 
4: Randomly append container  $c_0 \in C$  to  $\pi$ ,  $C \leftarrow C \setminus c_0$ 
5: while  $C \neq \emptyset$  do
6:   Sort all containers  $c \in C$  in non-decreasing order of  $\eta_c$ 
   maintaining the current I/O point (expression (3))
7:    $RCL :=$  Pick the first  $\alpha$  containers as the candidates of the  $RCL$ 
8:    $\pi :=$  Append one random container  $r \in RCL$  to the end of  $\pi$ 
   and replace its I/O point with the best one on this side
9:    $C \leftarrow C \setminus r$ 
10: Update I/O points using I/O Assignment and Timing ( $\pi$ )

```

6. Computational experiments

In this section, comprehensive computational experiments are carried out using the benchmark instances proposed in Vallada et al. (2023). Moreover, a new set of calibration instances is generated in order to validate and tune the parameters of the GRASP algorithm and the local search proposed. We calibrate the proposed methods with the Design Of Experiments (DOE) approach (Montgomery, 2019). All methods are coded in C# using Microsoft Visual Studio 2019 and .Net 6.0. Computation is randomly distributed to servers running an OpenStack virtualization platform. Virtual machines with 16 GBytes of RAM memory and four virtual cores running Windows 10 Enterprise 64 bits are employed. The servers running the virtual machines have two 18 core Intel Xeon Gold 5220 processors running at 2.2 GHz and 384 GBytes of RAM.

6.1. Benchmark instances

The testing set of instances comes from Vallada et al. (2023). It is divided into three subsets of small ($n \in \{5, 10\}$), medium ($n \in \{15, 20, 30, 40\}$) and large ($n \in \{50, 100, 150, 200\}$) for a total of 10 size combinations. The values for the rows (R), bays (B), tiers (T), I/O points on both sides, landside and seaside are $R = 10$, $B = 42$, $T = 4$, 6 I/O points on the landside and 10 I/O points on the seaside, respectively, based on the real yard sizes of Gharehgozli et al. (2014, 2019). The initial location of the crane is also set to $(x, y, T + 1)$ where x and y were randomly generated from 1 to R and from 0 to $B + 1$ respectively. Initially, the crane is located at the top, that is, the z coordinate is set to $T + 1$ (5 in this case). In Vallada et al. (2023), the locations of the I/O points were obtained in the following way: For 10 I/O seaside points, locations are $(x, 0, 1)$, where $x = \{1, 2, \dots, 10\}$. An extra bay 0 is considered only for the I/O points on the seaside. Moreover, the z value is set to 1 for all these I/O points. Regarding the 6 landside I/O points, locations are set to $(x, B + 1, 2)$, where $x = \{2, 3, \dots, 7\}$. Again, an extra bay (43 in this case) is considered only for I/O points on the landside. Moreover, the z value is set to 2 since the containers are delivered to a truck platform. Similarly to Vallada

et al. (2023), we also employ the extra small set of instances (small tight) with the same $n \in \{5, 10\}$ which was generated with only one and two I/O points on the landside and seashores respectively. In this case, the location of the I/O at the landside is $(4, B + 1, 2)$, and the location of I/O points on the seaside are $(4, 0, 1)$ and $(5, 0, 1)$. These small tight instances were generated in order to check the congestion at the I/O points. Finally, the speed of the crane in the gantry, trolley, and hoisting is constant (equal to 1) as there is no crane interference (Speer and Fischer, 2017). Additionally, a reshuffle may be needed in some cases, so 2 unit times per reshuffle (Res_{time}) are also included in the instances.

In all instances, the container data is randomly generated. The necessary information includes container type, yard location, number of reshuffles, time parameters, and weights. The container type is randomly generated from 1 to 4 for C_1 to C_4 . As regards the number of reshuffles, it is also randomly generated between 1 and T . The yard locations of containers, integer rows, bays and tiers are generated randomly as coordinates (x, y, z) , $x \leq R$, $y \leq B$ and $z \leq T$. The time parameters follow a uniform distribution in the range $U(0, \rho \times RoundTrip)$ for storage containers and $U(0, 0.4 \times RoundTrip)$ for retrieval containers, where $RoundTrip = n \times B$ is the maximum total distance for loading all container requests. The parameter ρ is set to three different values $\{0.1, 0.4, 0.7\}$, reflecting different levels of congestion at the I/O points. Regarding weights, two types, *Equal* (E) and *Non Equal* (NE), are used to penalize tardiness (w_c^R), and congestion (w_c^C) in the objective function (2). In *Equal* instances, all weights are set to 1, whereas for *Non Equal* instances, all weights are fixed to 3 for containers in the seaside and 1 for the landside respectively. Considering the three values of ρ , the two types of weight and the 12 size combinations, we have $3 \times 2 \times 12 = 72$ instance combinations. 10 instance replicates are generated for each combination for a final testing benchmark of 720 instances.

In this paper, we present a new set of calibration instances differentiating from the test instances to calibrate the proposed GRASP algorithms. In this case, the same procedure as above is used, but with different random seeds, to obtain a smaller calibration benchmark. More specifically, we have two replicates for each combination, for a total of 144 calibration instances.

6.2. Calibration of the proposed GRASP

The calibration of the proposed *GR1* and *GR2* consist of tuning the value of two parameters: β (size of the set with the best insertions for a given container) for the local search and α for the size of the RCL . Moreover, it is necessary to choose a deterministic rule in the constructive phase. To avoid overfitting in the final results, we employ the set of calibration instances detailed in the previous section. The different configurations for the proposed GRASP algorithms are run $R = 5$ independent times for each calibration instance with different random seeds used as a variance reduction technique (McGeoch, 1992). The stopping criterion is set to $n \times \ln n \times \text{Total I/O points} \times \tau$ milliseconds, where $\tau = 10$ for the calibration. The rationale is to give the GRASP more CPU time as the size of the instance grows.

The well known Analysis of the Variance (ANOVA) technique is used to analyze the results when calibrating the aforementioned parameters. The idea is to statistically check if the observed differences in the averages of the studied variants/levels of the factors are statistically significant (Montgomery, 2019). The response variable is the average Relative Percentage Deviation (RPD) following expression:

$$RPD = \frac{Method_{sol} - Best_{sol}}{Best_{sol}} \times 100 \quad (4)$$

where $Method_{sol}$ is the result obtained by the given GRASP configuration for the given instance, and $Best_{sol}$ is the best solution obtained for the same instance in all experimentation.

Being a parametric technique, ANOVA is most powerful when the following three hypotheses are satisfied: Normality, homoscedasticity,

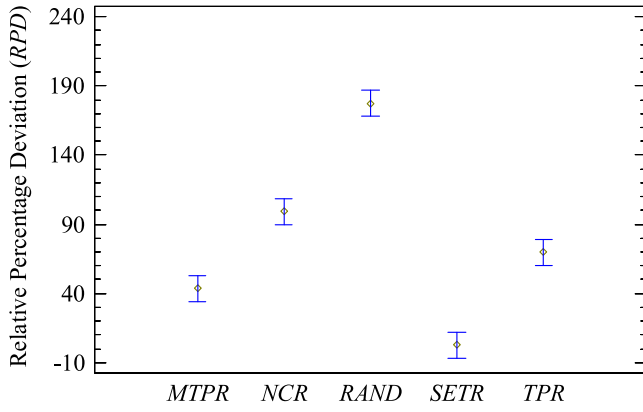


Fig. 4. Means and Tukey HSD intervals at the 95% confidence level for the proposed *SETR* rule, comparing rules from the literature and a baseline random rule.

and independence of the residuals, which all need to be checked. While no significant deviations were found, it has to be noted that ANOVA is particularly robust with respect to the first two hypotheses and the independence of the residuals is easy to control in computational experiments.

6.2.1. Rule selection for the constructive algorithm

We first validate if the proposed constructive heuristic (*SETR*) in Section 4 is competitive. To test this, we compare it with the best deterministic rules proposed in Vallada et al. (2023), namely *TPR*, *MTPR* and *NCR*. We employ a single factor ANOVA to study if the differences observed between these rules have a statistically significant effect on the dependent variable, which is the *RPD*. Moreover, acting as a baseline, a random rule (*RAND*) is also included. In which 5 solutions are generated using the random permutation of containers and the same I/O assignments in Section 4.2. The best of these 5 solutions is selected. Fig. 4 shows the means plot with Tukey's Honestly Significant Difference 95% confidence level intervals for the different rules. These intervals allow for multiple pairwise comparisons, meaning that whenever any two intervals overlap, the observed differences of the overlapped means are statistically insignificant. In this case, there are no overlaps. As expected, *RAND* performs the worst. The proposed *SETR* produces statistically lower *RPD* values and it is therefore selected as the constructive heuristic of the proposed GRASP methods.

6.2.2. β Selection of the local search

The local search proposed in Section 4.3 employs a single parameter β in order to set the number of solutions that are evaluated with the real objective function, that is, including the I/O assignment (Section 4.2). Four values are tested for $\beta = 1$ (*SETR*₁), 3 (*SETR*₃), 4 (*SETR*₄) and 5 (*SETR*₅). Fig. 5 shows the corresponding means plot for *GR1* and *GR2* where it can be seen that intervals for 3, 4, and 5 values overlap, however, all three are better than a beta value of 1. Since reducing β means that fewer solutions have to be evaluated using the real objective function, we select *SETR*₃ for speed, which is simply denoted as *SETR*₃ from now on.

6.2.3. Size of the RCL (α)

Finally, we calibrate the parameter α to set the size of the *RCL* in the GRASP algorithms. Values ranging from 1 to 7 are tested. Fig. 6 shows the corresponding means plot. As is typical in GRASP, a value of 1, the deterministic selection of the best candidate, indicates the worst results. Values 2 and 3 are the best values for *GR1*, and values 2, 3, and 4 are statically equivalent for *GR2* due to overlapping intervals. For simplicity, values 2 and 3 are selected for *GR1* and *GR2*, respectively.

6.3. Computational results

We now proceed with a comprehensive experimental evaluation of the proposed GRASP algorithms, now using the test benchmark comprising 720 instances in small, small tight, medium, and large sets. The GRASP algorithms are compared to rules *TPR*₁, *MTPR*₁ and *NCR*₁ proposed in Vallada et al. (2023), as well as the proposed *SETR*₁ method (i.e., the *SETR* constructive and the local search). Note that *SETR* is a deterministic heuristic used both as a standalone method as well as a starting solution for GRASP (line 3 of Algorithm 2). Furthermore, the randomized version of *SETR* has been adapted as the constructive method of the proposed GRASP (line 5 of Algorithm 2). Similar to the calibration, the GRASP algorithms are conducted for $R = 5$ independent runs and we use three levels of $\tau = \{10, 20, 30\}$ for the stopping criterion. In doing so, CPU times range from 0.24 s to 8.48 min from the smallest to the largest instances. This allows us to test the effect of additional CPU times on the proposed GRASP. The obtained solutions can be compared with the optimal solutions obtained by the mathematical models proposed in Vallada et al. (2023). We report the number of optimal solutions obtained by the studied methods for small and small tight instances in Tables 4 and 5 respectively. *GR1* and *GR2* obtain 74 (62%) and 80 (67%) optimal solutions for small instances and 88 (73%) and 91 (76%) for small tight instances, regardless of the CPU time used. As a matter of fact, they obtain the same results for these two sets of instances. However, the best rules are only able to obtain 64 optimal solutions for small instances (53%) and 49 for the small tight set (41%).

In order to compare the performance of the rules and the GRASP, the average *RPD* is now given in Tables 6 and 7 where we also report the average CPU times in seconds for each algorithm. *GR1* _{τ} and *GR2* _{τ} , $\tau = \{10, 20, 30\}$ refer to *GR1* and *GR2* using different CPU times as termination criteria according to the values of τ . For small instances, it can be observed that the proposed *SETR*₁ heuristic does not perform better than *NCR*₁ but it is much faster: 0.06 s vs. 7.50 s, on average. Regarding the GRASP algorithm, *GR1* outperforms all the heuristics with an *RPD* 3.5 times lower than the best constructive heuristic (*NCR*₁) while for *GR2*, 4.5 times lower, regardless of the value of τ . Regarding small tight instances, the proposed *SETR*₁ heuristic performs better than *NCR*₁ while being 125 times faster. In the worst case of $\tau = 10$, *GR1* improves the best heuristic, being 5.43 (for *GR2*, 5.75) times better than *SETR*₁ with practically negligible additional CPU Time (0.47 s vs. 0.06 s on average). Note that for these two sets, small and small tight, GRASP produces the same result regardless of the amount of CPU time.

For the medium instances in Table 8, the best performance is observed for *GR2* with $\tau = 30$. In all cases, the average *RPD* of the GRASP method is clearly better than the values for all the heuristics, regardless of the τ . Results also improve as additional CPU time is given. It has to be mentioned that the proposed *SETR*₁ is, in most cases, better than all the other tested existing approaches from the literature and with a fraction of the CPU time.

Last, for large instances, the performance comparison among methods is similar to that for medium instances. From Table 9, we can see that both GRASP methods outperform the state-of-the-art heuristics, with the newly proposed *GR2* method surpassing *GR1* by a wide margin. *GR2*₃₀ yields the best results, obtaining a 0.28% *RPD* on average. While its computational times are longer than the best existing *NCR*₁ method, we can consider *GR2*₁₀ instead, which has an average *RPD* of 0.86 vs. 11.16 of *NCR*₁, while using just 98.70 s on average vs. 125 s of *NCR*₁.

Fig. 7 shows the interaction between the number of containers for the instance and the average *RPD* of the algorithms. This interaction plot is obtained after using the ANOVA technique again, in this case, with a two factor ANOVA involving the two studied factors. This allows us to statistically observe the effect of the instance size and algorithms over the response variable. The GRASP method (here shown just as

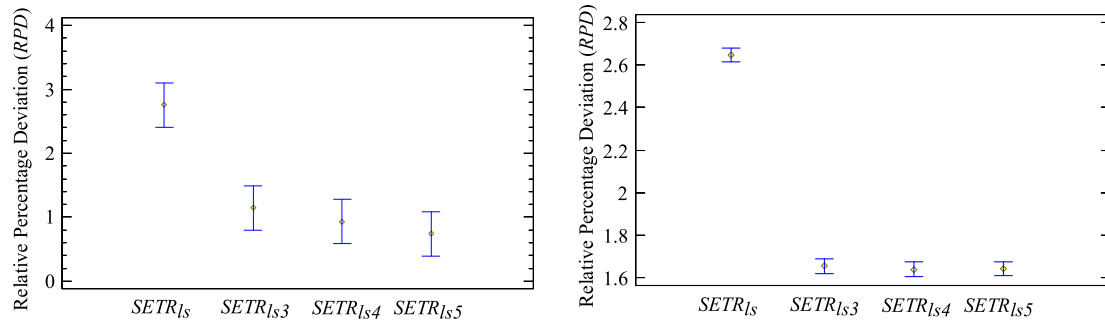


Fig. 5. Means and Tukey HSD intervals at the 95% confidence level for the tested β values of the proposed local search in GR1 (left) and GR2 (right).

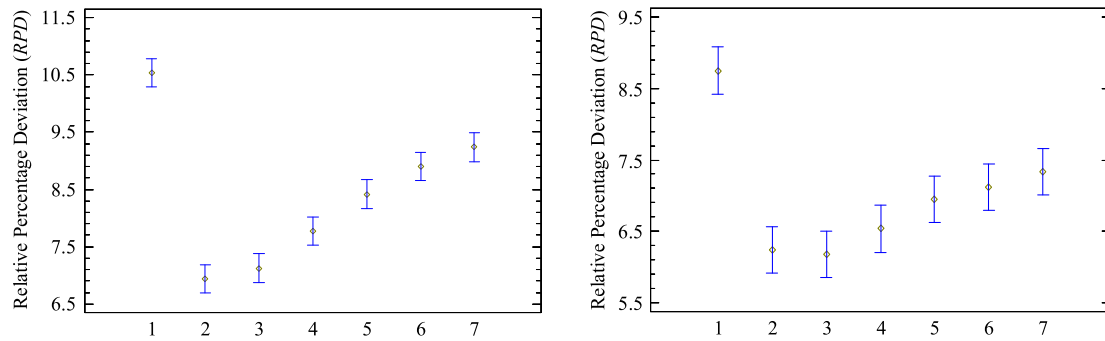


Fig. 6. Means and Tukey HSD intervals at the 95% confidence level for the tested α values of the proposed GR1 (left) and GR2 (right).

Table 4

Number of optimal solutions obtained by different algorithms for small instances with *Equal* (E) and *Non Equal* (NE) weights (w_j) grouped by parameter ρ .

n	w_j	ρ	#	TPR_{1s}	$MTPR_{1s}$	NCR_{1s}	$SETR_{1s}$	$GR1_{10}$	$GR2_{10}$	$GR1_{20}$	$GR2_{20}$	$GR1_{30}$	$GR2_{30}$
5	E	0.1	10	8	8	9	6	6	9	6	9	6	9
		0.4	10	8	6	10	9	10	10	10	10	10	10
		0.7	10	6	8	8	8	9	9	9	9	9	9
	NE	0.1	10	6	7	7	4	7	8	7	8	7	8
		0.4	10	8	7	9	6	7	8	7	8	7	8
		0.7	10	8	8	7	8	10	10	10	10	10	10
10	E	0.1	10	2	0	0	1	3	3	3	3	3	3
		0.4	10	0	1	3	1	5	5	5	5	5	5
		0.7	10	1	1	1	2	4	5	4	5	4	5
	NE	0.1	10	3	1	2	1	4	4	4	4	4	4
		0.4	10	3	2	2	1	3	3	3	3	3	3
		0.7	10	3	3	6	2	6	6	6	6	6	6
Total			120	56	52	64	49	74	80	74	80	74	80

Table 5

Number of optimal solutions obtained by different algorithms for small tight instances with *Equal* (E) and *Non Equal* (NE) weights (w_j) grouped by parameter ρ .

n	w_j	ρ	#	TPR_{1s}	$MTPR_{1s}$	NCR_{1s}	$SETR_{1s}$	$GR1_{10}$	$GR2_{10}$	$GR1_{20}$	$GR2_{20}$	$GR1_{30}$	$GR2_{30}$
5	E	0.1	10	3	3	7	9	10	10	10	10	10	10
		0.4	10	6	5	5	7	9	9	9	9	9	9
		0.7	10	7	6	5	6	9	10	9	10	9	10
	NE	0.1	10	6	5	5	3	8	8	8	8	8	8
		0.4	10	8	7	7	8	9	9	9	9	9	9
		0.7	10	7	7	8	7	8	8	8	8	8	8
10	E	0.1	10	1	0	2	2	8	8	8	8	8	8
		0.4	10	1	1	0	2	5	5	5	5	5	5
		0.7	10	3	1	3	2	7	7	7	7	7	7
	NE	0.1	10	0	0	2	0	4	6	4	6	4	6
		0.4	10	2	1	2	1	5	5	5	5	5	5
		0.7	10	1	2	3	2	6	6	6	6	6	6
Total			120	45	38	49	49	88	91	88	91	88	91

Table 6

Average *RPD* (%) over optimal solutions and CPU times (s) for small instances with *Equal* (E) and *Non Equal* (NE) weights (w_j) grouped by parameter ρ .

n	w_j	ρ	TPR_{1s}	$MTPR_{1s}$	NCR_{1s}	time	$SETR_{1s}$	time	$GR1_{10}$	$GR2_{10}$	time	$GR1_{20}$	$GR2_{20}$	time	$GR1_{30}$	$GR2_{30}$	time
5	E	0.1	0.96	0.25	1.30	5.00	0.73	0.05	0.73	0.14	1.29	0.73	0.14	2.58	0.73	0.14	3.86
		0.4	4.77	2.16	0.00	5.00	0.33	0.05	0.00	0.00	1.29	0.00	0.00	2.58	0.00	0.00	3.86
		0.7	5.31	2.76	0.76	5.00	2.23	0.05	0.13	0.13	1.29	0.13	0.13	2.58	0.13	0.13	3.86
	NE	0.1	0.35	0.26	0.26	5.00	1.71	0.06	0.22	0.12	1.29	0.22	0.12	2.58	0.22	0.12	3.86
		0.4	2.44	2.76	0.54	5.00	2.45	0.05	0.82	0.73	1.29	0.82	0.73	2.58	0.82	0.73	3.86
		0.7	2.37	2.37	2.31	5.00	0.76	0.05	0.00	0.00	1.29	0.00	0.00	2.58	0.00	0.00	3.86
10	E	0.1	3.27	4.73	4.41	10.00	2.99	0.06	0.54	0.41	3.68	0.54	0.40	7.37	0.54	0.40	11.05
		0.4	4.48	3.99	2.07	10.00	4.90	0.06	0.37	0.37	3.68	0.37	0.37	7.37	0.37	0.37	11.05
		0.7	4.69	8.22	4.06	10.00	2.32	0.06	0.87	0.49	3.68	0.87	0.41	7.37	0.87	0.41	11.05
	NE	0.1	2.11	7.04	2.54	10.00	3.96	0.06	1.39	1.33	3.68	1.39	1.23	7.37	1.39	1.23	11.05
		0.4	1.45	6.94	2.40	10.00	1.72	0.06	0.66	0.66	3.68	0.66	0.66	7.37	0.66	0.63	11.05
		0.7	2.88	4.29	3.24	10.00	4.23	0.07	1.13	1.13	3.68	1.13	1.13	7.37	1.13	1.13	11.05
Total average			2.92	3.81	1.99	7.50	2.36	0.06	0.57	0.46	2.48	0.57	0.44	4.97	0.57	0.44	7.46

Table 7

Average *RPD* (%) over optimal solutions and CPU times (s) for small tight instances with *Equal* (E) and *Non Equal* (NE) weights (w_j) grouped by parameter ρ .

n	w_j	ρ	TPR_{1s}	$MTPR_{1s}$	NCR_{1s}	time	$SETR_{1s}$	time	$GR1_{10}$	$GR2_{10}$	time	$GR1_{20}$	$GR2_{20}$	time	$GR1_{30}$	$GR2_{30}$	time
5	E	0.1	2.56	2.05	1.19	5.00	0.06	0.05	0.00	0.00	0.24	0.00	0.00	0.48	0.00	0.00	0.72
		0.4	2.22	3.71	2.52	5.00	1.05	0.06	0.10	0.10	0.24	0.10	0.10	0.48	0.10	0.10	0.72
		0.7	2.38	2.17	7.56	5.00	4.41	0.05	0.06	0.00	0.24	0.06	0.00	0.48	0.06	0.00	0.72
	NE	0.1	1.82	2.50	1.98	5.00	4.48	0.06	0.92	0.89	0.24	0.92	0.89	0.48	0.92	0.89	0.72
		0.4	0.83	2.15	3.68	5.00	1.90	0.06	0.45	0.45	0.24	0.45	0.45	0.48	0.45	0.45	0.72
		0.7	1.39	1.39	1.26	5.00	1.39	0.05	1.26	1.26	0.24	1.26	1.26	0.48	1.26	1.26	0.72
10	E	0.1	6.44	7.31	5.04	10.00	6.35	0.06	0.64	0.64	0.69	0.64	0.64	1.38	0.64	0.64	2.07
		0.4	6.74	8.72	12.06	10.00	4.16	0.05	0.84	0.80	0.69	0.84	0.80	1.38	0.84	0.80	2.07
		0.7	7.65	10.51	6.44	10.00	4.36	0.05	1.26	1.26	0.69	1.26	1.26	1.38	1.26	1.26	2.07
	NE	0.1	6.99	8.49	2.65	10.00	5.43	0.06	0.95	0.59	0.69	0.95	0.57	1.38	0.95	0.57	2.07
		0.4	5.08	10.41	3.63	10.00	5.75	0.06	1.19	1.19	0.69	1.19	1.19	1.38	1.19	1.19	2.07
		0.7	8.50	9.06	4.82	10.00	6.23	0.06	0.80	0.80	0.69	0.80	0.80	1.38	0.80	0.80	2.07
Total average			4.38	5.71	4.40	7.50	3.80	0.06	0.70	0.66	0.47	0.70	0.66	0.93	0.70	0.66	1.39

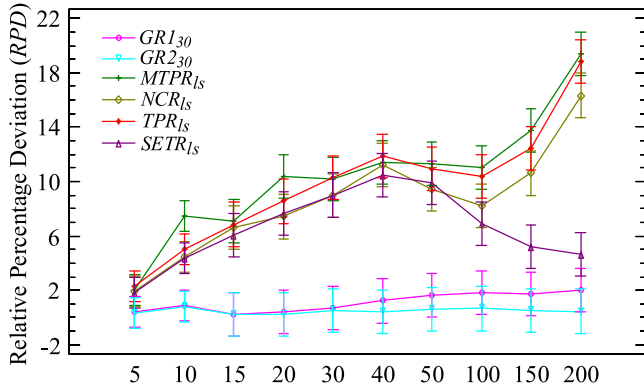
Table 8

Average *RPD* (%) over the best known solution and CPU times (s) for medium instances with *Equal* (E) and *Non Equal* (NE) weights (w_j) grouped by parameter ρ .

n	w_j	ρ	TPR_{1s}	$MTPR_{1s}$	NCR_{1s}	time	$SETR_{1s}$	time	GR^1_{10}	GR^2_{10}	time	GR^1_{20}	GR^2_{20}	time	GR^1_{30}	GR^2_{30}	time
15	E	0.1	6.75	6.46	5.57	15.00	4.81	0.07	0.72	0.69	6.50	0.72	0.69	13.00	0.72	0.50	19.50
		0.4	8.67	5.34	7.17	15.00	5.73	0.06	0.06	0.06	6.50	0.06	0.06	13.00	0.06	0.06	19.50
		0.7	6.99	6.22	6.41	15.00	4.08	0.07	0.10	0.10	6.50	0.10	0.10	13.00	0.10	0.10	19.50
	NE	0.1	3.52	5.03	2.01	15.00	4.17	0.06	0.32	0.28	6.50	0.31	0.28	13.00	0.31	0.28	19.50
		0.4	11.13	7.25	8.64	15.00	7.49	0.07	0.00	0.00	6.50	0.00	0.00	13.00	0.00	0.00	19.50
		0.7	4.01	12.19	9.71	15.00	10.01	0.07	0.04	0.04	6.50	0.04	0.04	13.00	0.04	0.04	19.50
Average			6.85	7.08	6.59	15.00	6.05	0.07	0.21	0.20	6.50	0.20	0.20	13.00	0.20	0.16	19.50
20	E	0.1	6.14	5.27	2.95	20.00	4.46	0.09	0.53	0.41	9.59	0.43	0.41	19.17	0.43	0.40	28.76
		0.4	7.95	5.92	7.87	20.00	4.55	0.08	0.06	0.01	9.59	0.02	0.00	19.17	0.02	0.00	28.76
		0.7	12.81	14.70	10.62	20.00	8.43	0.07	0.62	0.54	9.59	0.58	0.48	19.17	0.58	0.44	28.76
	NE	0.1	5.88	8.53	4.47	20.00	5.93	0.09	0.15	0.09	9.59	0.03	0.04	19.17	0.03	0.01	28.76
		0.4	8.71	11.87	6.72	20.00	10.11	0.10	0.24	0.21	9.59	0.15	0.14	19.17	0.04	0.11	28.76
		0.7	9.78	15.82	11.94	20.00	12.53	0.09	0.81	0.24	9.59	0.72	0.24	19.17	0.14	0.20	28.76
Average			8.54	10.35	7.43	20.00	7.67	0.09	0.40	0.25	9.59	0.32	0.22	19.17	0.21	0.19	28.76
30	E	0.1	7.10	8.84	8.57	30.00	5.09	0.14	0.42	0.46	16.33	0.26	0.30	32.65	0.22	0.11	48.98
		0.4	12.21	8.46	9.72	30.00	10.70	0.13	0.67	0.49	16.33	0.43	0.29	32.65	0.32	0.18	48.98
		0.7	11.49	10.80	12.32	30.00	7.74	0.14	0.53	0.55	16.33	0.12	0.40	32.65	0.03	0.34	48.98
	NE	0.1	7.68	8.55	4.01	30.00	9.20	0.13	1.08	0.79	16.33	0.81	0.50	32.65	0.73	0.39	48.98
		0.4	8.36	8.17	7.63	30.00	10.88	0.16	0.69	0.57	16.33	0.29	0.34	32.65	0.26	0.21	48.98
		0.7	14.96	16.31	11.64	30.00	10.46	0.15	0.69	0.22	16.33	0.58	0.19	32.65	0.52	0.08	48.98
Average			10.30	10.19	8.98	30.00	9.01	0.14	0.68	0.51	16.33	0.42	0.34	32.65	0.35	0.22	48.98
40	E	0.1	11.88	12.75	11.68	40.00	7.75	0.24	1.28	0.66	23.61	0.90	0.55	47.22	0.83	0.34	70.83
		0.4	16.66	10.41	14.26	40.00	11.02	0.24	0.73	1.15	23.61	0.43	0.74	47.22	0.32	0.26	70.83
		0.7	17.90	13.26	17.41	40.00	11.36	0.22	1.05	0.97	23.61	0.79	0.55	47.22	0.60	0.16	70.83
	NE	0.1	6.78	7.01	6.09	40.00	6.53	0.24	0.33	0.27	23.61	0.24	0.14	47.22	0.18	0.09	70.83
		0.4	8.45	11.89	7.67	40.00	10.01	0.22	0.76	0.70	23.61	0.51	0.44	47.22	0.40	0.07	70.83
		0.7	9.75	13.21	10.26	40.00	16.27	0.22	1.07	0.89	23.61	0.42	0.60	47.22	0.30	0.40	70.83
Average			11.90	11.42	11.23	40.00	10.49	0.23	0.87	0.77	23.61	0.55	0.50	47.22	0.44	0.22	70.83
Total average			9.40	9.76	8.56	26.25	8.30	0.13	0.54	0.43	14.01	0.37	0.31	28.01	0.30	0.20	42.02

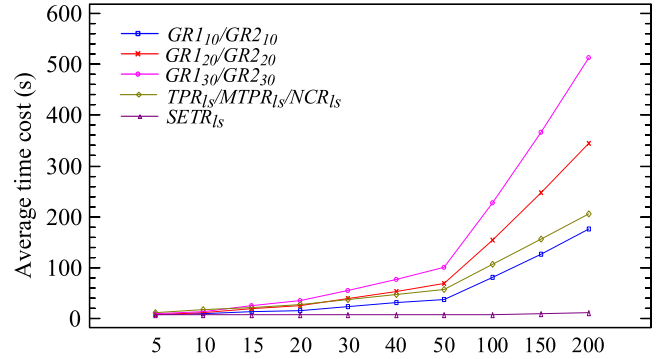
Table 9Average RPD over the best known solution and CPU times (s) for large instances with *Equal* (E) and *Non Equal* (NE) weights (w_j) grouped by parameter ρ .

n	w_j	ρ	TPR_{Is}	$MTPR_{Is}$	NCR_{Is}	time	$SETR_{Is}$	time	$GR1_{10}$	$GR2_{10}$	time	$GR1_{20}$	$GR2_{20}$	time	$GR1_{30}$	$GR2_{30}$	time
50	E	0.1	8.94	12.38	9.14	50.00	6.78	0.35	1.33	0.89	31.30	1.00	0.46	62.59	0.68	0.34	93.89
		0.4	12.59	9.98	13.21	50.00	11.86	0.31	1.22	0.86	31.30	0.75	0.41	62.59	0.56	0.33	93.89
		0.7	15.08	16.03	12.87	50.00	11.39	0.31	1.43	0.82	31.30	1.11	0.57	62.59	0.94	0.44	93.89
	NE	0.1	6.33	6.02	4.51	50.00	6.47	0.40	1.02	1.00	31.30	0.74	0.56	62.59	0.50	0.26	93.89
		0.4	9.35	9.19	7.50	50.00	9.55	0.33	1.00	1.23	31.30	0.70	0.71	62.59	0.30	0.53	93.89
		0.7	13.54	14.18	9.46	50.00	13.30	0.37	1.08	1.54	31.30	0.63	0.57	62.59	0.51	0.23	93.89
	Average		10.97	11.30	9.45	50.00	9.89	0.35	1.18	1.06	31.30	0.82	0.55	62.59	0.58	0.36	93.89
100	E	0.1	8.77	10.73	7.35	100.00	3.66	1.03	1.63	0.48	73.68	1.26	0.23	147.37	1.23	0.09	221.05
		0.4	13.55	14.26	12.78	100.00	6.75	1.28	2.54	0.85	73.68	1.99	0.53	147.37	1.68	0.10	221.05
		0.7	18.04	16.71	15.75	100.00	10.90	1.07	1.45	1.69	73.68	1.06	0.84	147.37	0.98	0.65	221.05
	NE	0.1	6.60	7.44	4.70	100.00	4.38	1.15	0.32	0.42	73.68	0.32	0.27	147.37	0.14	0.17	221.05
		0.4	9.71	9.17	5.26	100.00	7.12	1.11	0.88	0.82	73.68	0.69	0.46	147.37	0.50	0.40	221.05
		0.7	5.74	8.07	3.57	100.00	8.64	1.30	2.03	1.68	73.68	1.35	1.03	147.37	1.18	0.78	221.05
	Average		10.40	11.06	8.23	100.00	6.91	1.16	1.47	0.99	73.68	1.11	0.56	147.37	0.95	0.36	221.05
150	E	0.1	9.01	9.84	9.03	150.00	3.15	2.87	1.91	0.21	120.26	1.64	0.10	240.51	1.62	0.00	360.77
		0.4	13.38	13.35	13.27	150.00	4.55	3.06	1.90	0.40	120.26	1.65	0.01	240.51	1.60	0.01	360.77
		0.7	19.18	18.91	18.29	150.00	8.54	2.77	2.01	1.37	120.26	1.29	1.16	240.51	0.82	0.52	360.77
	NE	0.1	8.54	9.26	6.88	150.00	3.14	2.47	0.97	0.80	120.26	0.47	0.50	240.51	0.30	0.32	360.77
		0.4	12.83	16.91	7.44	150.00	4.22	2.97	1.05	0.78	120.26	0.57	0.28	240.51	0.48	0.17	360.77
		0.7	11.75	14.23	8.79	150.00	7.63	2.65	1.19	1.08	120.26	0.78	0.70	240.51	0.52	0.64	360.77
	Average		12.45	13.75	10.62	150.00	5.20	2.80	1.51	0.77	120.26	1.07	0.46	240.51	0.89	0.28	360.77
200	E	0.1	10.96	12.53	9.67	200.00	2.26	4.50	2.72	0.45	169.55	2.40	0.15	339.09	2.14	0.00	508.64
		0.4	21.22	19.34	17.03	200.00	4.13	4.45	2.65	0.48	169.55	2.29	0.14	339.09	1.99	0.00	508.64
		0.7	29.34	28.70	27.58	200.00	8.11	5.15	2.75	0.74	169.55	2.12	0.47	339.09	1.60	0.14	508.64
	NE	0.1	10.46	12.64	9.25	200.00	1.39	6.43	0.57	0.29	169.55	0.31	0.16	339.09	0.22	0.03	508.64
		0.4	21.40	24.92	17.50	200.00	4.18	6.23	1.34	0.55	169.55	0.91	0.33	339.09	0.33	0.20	508.64
		0.7	19.74	18.06	16.94	200.00	7.96	4.81	1.17	1.21	169.55	0.61	0.62	339.09	0.28	0.41	508.64
	Average		18.85	19.36	16.33	200.00	4.67	5.26	1.86	0.62	169.55	1.44	0.31	339.09	1.09	0.13	508.64
Total average			13.17	13.87	11.16	125.00	6.67	2.39	1.51	0.86	98.70	1.11	0.47	197.39	0.88	0.28	296.09

**Fig. 7.** Interaction and Tukey HSD intervals at the 95% confidence level between the number of containers (n) and RPD for the algorithms.

$GR1_{30}$ and $GR2_{30}$ for simplicity) demonstrates robust performance, that is, it always obtains the lowest RPD regardless of the size of the instance. The interaction shows an interesting effect for instances of 50 containers or more. While the methods proposed in Vallada et al. (2023) quickly deteriorate, that is, the performance clearly gets worse as the size of the instance increases, the proposed $SETR_{Is}$ heuristic follows an opposite trend with respect to the remaining deterministic heuristics, since the local search proposed is much better, from both points of view efficiency and effectiveness, than the local search used in Vallada et al. (2023), showing its strength, particularly for bigger problems.

Fig. 8 shows the average CPU times (in seconds) for the algorithms as a function of the number of containers n . Note that only one of the rules from Vallada et al. (2023) is pictured (NCR_{Is}) since the CPU times for the three rules are the same. It can be observed that the proposed $SETR_{Is}$ heuristic needs very little CPU time regardless of the size of the

**Fig. 8.** Average time (in seconds) according to the number of containers (n).

instance. For the rest of the algorithms, the time significantly increases with 50 or more containers. The fastest tested GRASP, with $\tau = 10$, is even faster than the NCR_{Is} rule, obtaining much better results as well. The other two versions of the GRASP need more time, while result in even better RPD values.

6.4. Sensitivity analysis

Even though GRASP has demonstrated to outperform the other approaches, it is still worth evaluating how it performs using various I/O assignments and weight settings.

6.4.1. Sensitivity analysis on I/O assignments

We compare the proposed $GR2$, denoted as $GR_{AssignIO}$, with two variants that replace I/O assignments with pre-fixed I/O alternatives from Section 4.2, which are referred to as GR_{RandIO} and $GR_{NearestIO}$, respectively. These three methods are tested on the same instances for CPU time $\tau = 10$. Fig. 9 shows the interaction and Tukey HSD intervals

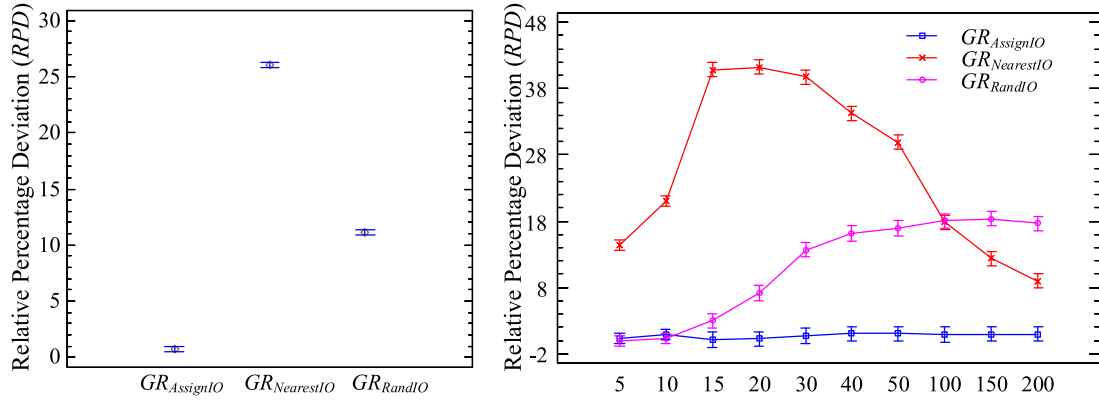


Fig. 9. Interaction and Tukey HSD intervals of RPD on the objective function at the 95% confidence level between the number of containers (n) and for the three I/O assignment alternatives.

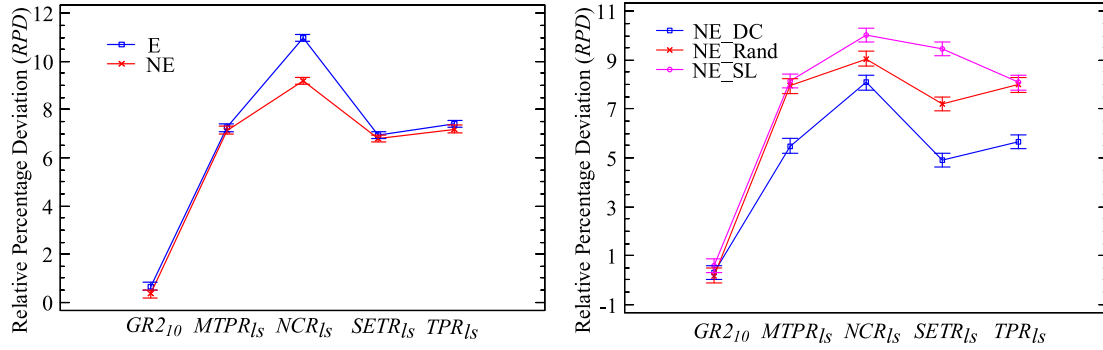


Fig. 10. Interaction and Tukey HSD intervals for the tested methods under equal (E) and non-equal (NE) weight settings (left) and under 3 kinds of non-equal weight settings (rights) at the 95% confidence level.

of RPD at the 95% confidence level. It can be seen that $GR_{AssignIO}$ significantly outperforms the other methods. Random I/O points result in worse performance as the number of containers increases, while the nearest I/O point performs exactly the opposite. The results align with the intuition that dynamic assignments help balance the I/O points, which helps in decreasing congestion and delay. Similar observations indicate that the proposed I/O assignment also results in a lower makespan. Note that the corresponding results are not detailed due to space limitations.

6.4.2. Sensitivity analysis of objective function weights

We now analyze the sensitivity of the proposed method across the different settings of weights. Four groups of weight settings are tested, including an equal (E) and three non-equal sets (NE). Each set consists of the 720 instances described in Section 6.1 but with different weight settings. For the E set, all containers are assigned a weight of 1. For NE sets, three different distributions of weights are considered:

1. NE_DC: Weights for Delay and congestion. Two weight pairs (1, 3) and (3, 1) are used for the delay (w_c^R) and congestion (w_c^C).
2. NE_SL: Weights for seaside and landside containers. Weight pairs (1, 3) and (3, 1) are fixed to the seaside ($c \in \{1, 3\}$) and landside ($c \in \{2, 4\}$) containers with identical weights for delay and congestion.
3. NE_Rand: Random weights. All weights are randomly set between 1 and 3.

One more ANOVA experiment is performed with $GR2$ on factors of equal/non-equal settings and different non-equal settings. Fig. 10 (left) presents the means and Tukey HSD intervals at the 95% confidence level for the tested methods between equal and non-equal settings. For time $\tau = 10$, the proposed $GR2$ significantly outperforms all other

methods and there is no relevant difference between equal and non-equal weights. A similar behavior can be observed among TPR_{ls} , $MTPR_{ls}$ and $SETR_{ls}$, being NCR_{ls} better for NE weights. As shown in Fig. 10 (right), $GR2$ continues to clearly outperform all other methods for the three non-equal weight settings. The proposed GRASP method is observed to be robust across all the tested weight settings.

7. Conclusions and future research

This paper deals with a more realistic version of the yard crane scheduling problem and contributes to bridging the gap between academic research and real life problems in yard crane scheduling. This version considers I/O congestion at the blocks in the yard of a container terminal. Furthermore, I/O points are limited in number. We consider that there are different types of containers, some of them are retrieved from the yard (to sea/land), and the other ones are stored in the yard (from sea/land). All containers incorporate parameters such as release times, due dates, and truck/train arrival times.

In order to solve the problem we have presented a very fast constructive heuristic. A local search using a proxy function that results in very fast neighborhood evaluations has also been proposed. Finally, these algorithms are incorporated into two GRASP algorithms, $GR1$ and $GR2$. Comprehensive calibration and computational testings show that the proposed deterministic heuristic and local search procedures ($SETR_{ls}$) generate results that are comparable to existing heuristics in small instances but orders of magnitude faster. For medium and large instances, they are clearly better while also being many times faster. Performance improves further with the proposed GRASP methods. Even with the shortest controllable CPU time, both GRASP methods are significantly better and faster than current heuristics, with $GR2$ consistently performing better at all CPU times.

The salient conclusion is that the proposed $SETR_{ls}$ is a better alternative when compared to the state-of-the-art solutions in practical situations where the available time for computation is very limited. If CPU time is less of a concern, the proposed GRASP methods obtain much better results.

For future research, we would like to explore local search in the assignment of I/O points in conjunction with the container sequence or in serialization with a variable neighborhood scheme. Further closing the gap between theory and practice, by bringing the problem even closer to reality, is a promising avenue of research.

CRedit authorship contribution statement

Hongtao Wang: Writing – original draft, Validation, Software, Methodology, Data curation, Conceptualization. **Fulgencia Villa:** Writing – review & editing, Supervision, Resources, Methodology. **Eva Vallada:** Writing – review & editing, Supervision, Resources, Methodology. **Rubén Ruiz:** Writing – review & editing, Supervision, Resources, Funding acquisition.

Data availability

Data will be made available online at: <https://github.com/HWangUPV/YCSP>.

Acknowledgments

The authors would like to express their most sincere gratitude to the anonymous referees and their insightful comments which have resulted in a much stronger manuscript. Hongtao Wang is sponsored by the Chinese Scholarship Council (No. 202006890026). The authors are partially supported by the Spanish Ministry of Science and Innovation under the project “OPRES-Realistic Optimization in Problems in Public Health”(No. PID2021-124975OB-I00) partially financed with FEDER funds and by the Agència Valenciana de la Innovació (AVI) under the project “ireves (innovación en vehículos de emergencia sanitaria): una herramienta inteligente de decisión” (No. INNACC/2021/26, interested readers can visit <http://ireves.upv.es>).

Appendix A. Supplementary data

Supplementary material related to this article can be found online at <https://doi.org/10.1016/j.cor.2024.106853>.

References

Cao, J.X., Lee, D.H., Chen, J.H., Shi, Q., 2010. The integrated yard truck and yard crane scheduling problem: Benders' decomposition-based methods. *Transp. Res. E* 46 (3), 344–353. <http://dx.doi.org/10.1016/j.tre.2009.08.012>.

Chang, D., Jiang, Z., Yan, W., He, J., 2011. Developing a dynamic rolling-horizon decision strategy for yard crane scheduling. *Adv. Eng. Inform.* 25 (3), 485–494. <http://dx.doi.org/10.1016/j.aei.2011.02.003>.

Chen, L., Langevin, A., 2011. Multiple yard cranes scheduling for loading operations in a container terminal. *Eng. Optim.* 43 (11), 1205–1221. <http://dx.doi.org/10.1080/0305215X.2010.548865>.

Chu, F., He, J., Zheng, F., Liu, M., 2019. Scheduling multiple yard cranes in two adjacent container blocks with position-dependent processing times. *Comput. Ind. Eng.* 136, 355–365. <http://dx.doi.org/10.1016/j.cie.2019.07.013>.

Dorndorf, U., Schneider, F., 2010. Scheduling automated triple cross-over stacking cranes in a container yard. *OR Spectrum* 32 (3), 617–632. <http://dx.doi.org/10.1007/s00291-010-0206-3>.

Feo, T., Resende, M., 1995. Greedy randomized adaptive search procedures. *J. Global Optim.* 6, 109–133. <http://dx.doi.org/10.1007/BF01096763>.

Galle, V., Barnhart, C., Jaillet, P., 2018. Yard crane scheduling for container storage, retrieval, and relocation. *European J. Oper. Res.* 271 (1), 288–316. <http://dx.doi.org/10.1016/j.ejor.2018.05.007>.

Gharehgozli, A.H., Laporte, G., Yu, Y., de Koster, R., 2015. Scheduling twin yard cranes in a container block. *Transp. Sci.* 49 (3), 686–705. <http://dx.doi.org/10.1287/trsc.2014.0533>.

Gharehgozli, A.H., Roy, D., De Koster, R., 2016. Sea container terminals: New technologies and OR models. *Marit. Econ. Logist.* 18 (2), 103–140. <http://dx.doi.org/10.1057/mel.2015.3>.

Gharehgozli, A.H., Vernooij, F.G., Zaerpour, N., 2017. A simulation study of the performance of twin automated stacking cranes at a seaport container terminal. *European J. Oper. Res.* 261 (1), 108–128. <http://dx.doi.org/10.1016/j.ejor.2017.01.037>.

Gharehgozli, A., Yu, Y., de Koster, R., Du, S., 2019. Sequencing storage and retrieval requests in a container block with multiple open locations. *Transp. Res. E* 125, 261–284. <http://dx.doi.org/10.1016/j.tre.2019.03.008>.

Gharehgozli, A.H., Yu, Y., de Koster, R., Udding, J.T., 2014. An exact method for scheduling a yard crane. *European J. Oper. Res.* 235 (2), 431–447. <http://dx.doi.org/10.1016/j.ejor.2013.09.038>.

He, J., Chang, D., Mi, W., Yan, W., 2010. A hybrid parallel genetic algorithm for yard crane scheduling. *Transp. Res. E* 46 (1), 136–155. <http://dx.doi.org/10.1016/j.tre.2009.07.002>.

He, J., Huang, Y., Yan, W., 2015a. Yard crane scheduling in a container terminal for the trade-off between efficiency and energy consumption. *Adv. Eng. Inform.* 29 (1), 59–75. <http://dx.doi.org/10.1016/j.aei.2014.09.003>.

He, J., Huang, Y., Yan, W., Wang, S., 2015b. Integrated internal truck, yard crane and quay crane scheduling in a container terminal considering energy consumption. *Expert Syst. Appl.* 42 (5), 2464–2487. <http://dx.doi.org/10.1016/j.eswa.2014.11.016>.

Hsu, H.P., Tai, H.H., Wang, C.N., Chou, C.C., 2021. Scheduling of collaborative operations of yard cranes and yard trucks for export containers using hybrid approaches. *Adv. Eng. Inform.* 48, 101292. <http://dx.doi.org/10.1016/j.aei.2021.101292>.

Hu, Z.H., Sheu, J.B., Luo, J.X., 2016. Sequencing twin automated stacking cranes in a block at automated container terminal. *Transp. Res. C* 69, 208–227. <http://dx.doi.org/10.1016/j.trc.2016.06.004>.

Jung, D.H., Park, Y.M., Lee, B.K., Kim, K.H., Ryu, K.R., 2006. A quay crane scheduling method considering interference of yard cranes in container terminals. In: Gelbukh, A., Reyes-Garcia, C.A. (Eds.), *MICAI 2006: Advances in Artificial Intelligence*. Springer, Berlin, Heidelberg, pp. 461–471. http://dx.doi.org/10.1007/11925231_43.

Kaveshgar, N., Huynh, N., 2015. Integrated quay crane and yard truck scheduling for unloading inbound containers. *Int. J. Prod. Econ.* 159, 168–177. <http://dx.doi.org/10.1016/j.ijpe.2014.09.028>.

Kim, K.H., Kim, K.Y., 1999. An optimal routing algorithm for a transfer crane in port container terminals. *Transp. Sci.* 33 (1), 17–33. <http://dx.doi.org/10.1287/trsc.33.1.17>.

Kim, K.H., Park, Y.-M., 2004. A crane scheduling method for port container terminals. *European J. Oper. Res.* 156 (3), 752–768. [http://dx.doi.org/10.1016/S0377-2217\(03\)00133-4](http://dx.doi.org/10.1016/S0377-2217(03)00133-4).

Kizilay, D., Eliyi, D.T., 2021. A comprehensive review of quay crane scheduling, yard operations and integrations thereof in container terminals. *Flex. Serv. Manuf. J.* 33 (1), 1–42. <http://dx.doi.org/10.1007/s10696-020-09385-5>.

Lee, D.H., Jin, J.G., Chen, J.H., 2011. Integrated bay allocation and yard crane scheduling problem for transshipment containers. *Transp. Res. Rec. J. Transp. Res. Board* 2222 (1), 63–71. <http://dx.doi.org/10.3141/2222-08>.

Li, W., Goh, M., Wu, Y., Petering, M., de Souza, R., Wu, Y., 2012. A continuous time model for multiple yard crane scheduling with last minute job arrivals. *Int. J. Prod. Econ.* 136 (2), 332–343. <http://dx.doi.org/10.1016/j.ijpe.2011.12.020>.

Li, W., Wu, Y., Petering, M., Goh, M., Souza, R.d., 2009. Discrete time model and algorithms for container yard crane scheduling. *European J. Oper. Res.* 198 (1), 165–172. <http://dx.doi.org/10.1016/j.ejor.2008.08.019>.

Liang, C.J., Chen, M., Gen, M., Jo, J., 2014. A multi-objective genetic algorithm for yard crane scheduling problem with multiple work lines. *J. Intell. Manuf.* 25 (5), 1013–1024. <http://dx.doi.org/10.1007/s10845-013-0792-4>.

Lim, A., Rodrigues, B., Xiao, F., Zhu, Y., 2004. Crane scheduling with spatial constraints. *Naval Res. Logist.* 51 (3), 386–406. <http://dx.doi.org/10.1002/nav.10123>.

McGeoch, C., 1992. Analyzing algorithms by simulation: variance reduction techniques and simulation speeds. *ACM Comput. Surv.* 24 (2), 195–212. <http://dx.doi.org/10.1145/130844.130853>.

Meade, R., 2022. Lloyd's List One Hundred Ports 2022. Technical Report, Maritime Insights & Intelligence Limited, London, URL: <https://lloydslist.maritimeintelligence.informa.com/one-hundred-container-ports-2022>.

Montgomery, D.C., 2019. Design and Analysis of Experiments, tenth ed. URL: <https://books.google.es/books?id=Py7bDgAAQBAJ>.

Narasimhan, A., Palekar, U.S., 2002. Analysis and algorithms for the transtainer routing problem in container port operations. *Transp. Sci.* 36 (1), 63–78. <http://dx.doi.org/10.1287/trsc.36.1.63.576>.

Ng, W.C., Mak, K.L., 2005. Yard crane scheduling in port container terminals. *Appl. Math. Model.* 29 (3), 263–276. <http://dx.doi.org/10.1016/j.apm.2004.09.009>.

Resende, M., Ribeiro, C., 2016. Optimization by GRASP: Greedy Randomized Adaptive Search Procedures. Springer New York, URL: <https://books.google.es/books?id=B7LpjWEACAAJ>.

Saini, S., Roy, D., de Koster, R., 2017. A stochastic model for the throughput analysis of passing dual yard cranes. *Comput. Oper. Res.* 87, 40–51. <http://dx.doi.org/10.1016/j.cor.2017.05.012>.

- Sirimanne, S.N., Hoffman, J., Juan, W., Asariotis, R., Assaf, M., Ayala, G., Benamara, H., Chantrel, D., Hoffmann, J., Premti, A., et al., 2022. Review of Maritime Transport 2022. Navigating Stormy Waters. Technical Report, United Nations Conference on Trade and Development (UNCTAD), Geneva, <http://dx.doi.org/10.18356/9789210021470>.
- Speer, U., Fischer, K., 2017. Scheduling of different automated yard crane systems at container terminals. *Transp. Sci.* 51 (1), 305–324. <http://dx.doi.org/10.1287/trsc.2016.0687>.
- Vallada, E., Belenguer, J.M., Villa, F., Alvarez-Valdes, R., 2023. Models and algorithms for a yard crane scheduling problem in container ports. *European J. Oper. Res.* 309 (2), 910–924. <http://dx.doi.org/10.1016/j.ejor.2023.01.047>.
- Vis, I.F., Carlo, H.J., 2010. Sequencing two cooperating automated stacking cranes in a container terminal. *Transp. Sci.* 44 (2), 169–182. <http://dx.doi.org/10.1287/trsc.1090.0298>.
- Vis, I.F., Roodbergen, K.J., 2009. Scheduling of container storage and retrieval. *Oper. Res.* 57 (2), 456–467. <http://dx.doi.org/10.1287/opre.1080.0621>.
- Yang, X.M., Jiang, X.J., 2020. Yard crane scheduling in the ground trolley-based automated container terminal. *Asia-Pac. J. Oper. Res.* 37 (2), 2050007. <http://dx.doi.org/10.1142/S0217595920500074>.
- Zheng, F., Man, X., Chu, F., Liu, M., Chu, C., 2018. Two yard crane scheduling with dynamic processing time and interference. *IEEE Trans. Intell. Transp. Syst.* 19 (12), 3775–3784. <http://dx.doi.org/10.1109/TITS.2017.2780256>.
- Zheng, F., Man, X., Chu, F., Liu, M., Chu, C., 2019. A two-stage stochastic programming for single yard crane scheduling with uncertain release times of retrieval tasks. *Int. J. Prod. Res.* 57 (13), 4132–4147. <http://dx.doi.org/10.1080/00207543.2018.1516903>.
- Zhou, C., Lee, B.K., Li, H., 2020. Integrated optimization on yard crane scheduling and vehicle positioning at container yards. *Transp. Res. E* 138, 101966. <http://dx.doi.org/10.1016/j.tre.2020.101966>.